

Ödev 05: Bozulamayan Bir Nesne

Kuralları, çağıranlardan dikkatli olmalarını rica eden bir yorumla değil, derleyici ve kurucu tarafından uygulanan bir sınıf kurun. Sonra onu kırmayı deneyin ve kıramadığınızı gösterin.

Görev

`Booking.java` dosyasını klasik biçimde oluşturun: bir `public class Booking` ve içinde `public static void main(String[] args)`.

İçinde şunları tutan bir `Reservation` sınıfı yazın:

- yolcu adı
- koltuk sırası numarası
- koltuk etiketlerinden oluşan bir liste, örneğin `["A", "B"]`

Garanti Etmesi Gereken Kurallar

Bir `Reservation` asla şu kurallardan birini bozan bir durumda var olamaz:

1. Yolcu adı asla null ve asla boş değildir
2. Sıra numarası her zaman 1 veya daha büyüktür
3. Her zaman en az bir koltuk vardır
4. Koltuk listesi nesnenin dışındaki hiç kimse tarafından değiştirilemez, **listeyi içeri veren dahil**

Dördüncü kural ilginç olanı ve iki yarısı var. Kurucunuzu çağırdıktan sonra çağıranın elinde hâlâ ne kaldığını düşünün.

Kanıtlayın

`main` içinde şunların her birini deneyin ve ne olduğunu yazdırın:

- null yolcu
- boş yolcu adı
- sıra `0`
- boş koltuk listesi
- içeri verdiğiniz listeye bir referans saklayıp sonradan ona ekleme yapmak
- getter'ınızın geri verdiği listede `add` çağırarak

Altı denemenin hepsi başarısız olmalı ya da hiçbir etki yaratmamalı. Çıktınız, nesnenin her seferinde kendini savunduğunu açıkça göstermeli.

Ayrıca İçermeli

- En az bir statik fabrika metodu, artı bunun kurucunun vermediği neyi verdiğini söyleyen bir yorum
- Rezervasyon yazdırmayı okunaklı kılan bir `toString()` geçersiz kılması

Kabul Kriterleri

- ☐ `java Booking.java` ile çalışıyor
- ☐ `public static void main(String[] args)` içeren `public class Booking`
- ☐ `Reservation` içindeki her alan `private` ve `final`
- ☐ Dört kuralın hepsi, herhangi bir alan atanmadan önce kurucuda uygulanıyor
- ☐ Kurulumdan sonra çağırının özgün listesini değiştirmek rezervasyonu değiştirmiyor
- ☐ Getter'ın döndürdüğü listeyi değiştirmek ya başarısız oluyor ya da rezervasyonu değiştirmiyor
- ☐ Bir statik fabrika var ve gerekçesini açıklayan bir yorum taşıyor
- ☐ `toString()` geçersiz kılınmış

İleri Seviye

Koltuk listesini değişmez biri yerine düz bir `ArrayList` alanı yapın ve dördüncü kuralı yine de çalıştırın. Sonra iki yaklaşımı karşılaştıran bir yorum yazın: her biri neye mal oluyor ve bu bedel hangi tarafa düşüyor, kurucuya mı yoksa getter'a yapılan her çağrıya mı?

İpucu, dördüncü kural elinizden kaçıyorsa

İki ayrı sızıntı var ve birini kapatmak diğerini kapatmıyor.

Birincisi: `new Reservation(..., seats)` çağrısından sonra o listeye tam olarak kim hâlâ bir referans tutuyor?

İkincisi: `reservation.seats()` çağrısından sonra az önce dışarı ne verdiniz?

`List.copyOf` ikisine birden çözüm, ve nedenini anlamak alıştırmamanın amacı.